

Fredrik Hammarberg

Optimizing Weak Reference Processing in the JVM Z Garbage Collector

*A Performance Analysis and Optimization of the ZGC
Reference Processing Pipeline*



UPPSALA
UNIVERSITET

Abstract
PLACEHOLDER

Acknowledgments

I would like to express my sincere gratitude to my supervisor for their guidance and support throughout this thesis work. Their expertise in JVM internals and garbage collection algorithms has been invaluable in shaping this research.

Special thanks to the Department of Information Technology at Uppsala University for providing the resources and environment necessary to conduct this research.

Finally, I would like to thank my family and friends for their patience and encouragement throughout my studies.

Uppsala, Sweden, June 2026

Fredrik Hammarberg

Contents

Acknowledgments	iii
1 Introduction	9
1.1 Motivation	9
1.2 Research Questions	9
1.3 Contributions	9
1.4 Thesis Structure	9
2 Background	10
2.1 Garbage Collection in the JVM	10
2.1.1 Basic Concepts	10
2.1.2 Stop-The-World vs Concurrent Collection	10
2.2 G1 Garbage Collector	10
2.2.1 Marking and Collection	10
2.3 Z Garbage Collector	10
2.3.1 Generational ZGC	10
2.4 Java Reference Types	10
2.4.1 WeakReference and ReferenceQueue	10
2.5 Reference Processing Pipeline	10
2.5.1 Discovery	10
2.5.2 Processing	10
2.5.3 Enqueue	10
2.6 Related Work	10
2.7 Summary	10
3 Method	11
3.1 Implementation Approach	11
3.1.1 ZGC Modifications	11
3.1.2 G1GC Modifications	11
3.2 Optimization Strategies	11
3.2.1 Null-Queue Skip Optimization	11
3.2.2 Data Structure Optimizations	11
3.2.3 Synchronization Optimizations	11
3.3 Benchmarking Methodology	11
3.3.1 Synthetic Benchmarks	11
3.3.2 Real-World Benchmarks	11
3.4 Experimental Setup	11
3.4.1 Hardware Configuration	11

3.4.2	Software Configuration	11
3.4.3	Measurement Methodology	11
4	Results	12
4.1	Synthetic Benchmark Results	12
4.1.1	Baseline Performance	12
4.1.2	Null-Queue Optimization Impact	12
4.1.3	Data Structure Impact	12
4.2	Caffeine Benchmark Results	12
4.2.1	Cache Operations Performance	12
4.2.2	GC Pause Time Analysis	12
4.3	Statistical Analysis	12
4.4	Summary of Findings	12
5	Discussion	13
5.1	Interpretation of Results	13
5.1.1	Performance Improvements	13
5.1.2	Trade-offs	13
5.2	Implications for JVM Development	13
5.2.1	Applicability to Other Collectors	13
5.2.2	Integration Challenges	13
5.3	Limitations	13
5.3.1	Benchmark Coverage	13
5.3.2	Correctness Considerations	13
5.3.3	Generational Complexity	13
5.4	Comparison with Related Work	13
5.5	Summary	13
6	Conclusion	14
6.1	Summary of Contributions	14
6.2	Answers to Research Questions	14
6.3	Future Work	14
6.3.1	Further Optimizations	14
6.3.2	Broader Evaluation	14
6.3.3	Integration with Mainline OpenJDK	14
6.4	Closing Remarks	14

List of Tables

List of Figures

1. Introduction

As the Java programming language moves further away from end-users and into the realm of large-scale enterprise applications, the performance of the Java Virtual Machine (JVM) becomes increasingly critical. Especially when it comes to performance on large heaps since many enterprise applications utilize large amounts of memory. The Z Garbage Collector (ZGC) is a low-latency garbage collector designed to handle large heaps with minimal pause times. However, the processing of weak references in ZGC has been identified as a potential bottleneck, particularly in applications that heavily utilize weak references. This thesis aims to analyze the performance of weak reference processing in ZGC and explore potential optimizations to improve its efficiency.

1.1 Motivation

1.2 Research Questions

1.3 Contributions

1.4 Thesis Structure

2. Background

2.1 Garbage Collection in the JVM

2.1.1 Basic Concepts

2.1.2 Stop-The-World vs Concurrent Collection

2.2 G1 Garbage Collector

2.2.1 Marking and Collection

2.3 Z Garbage Collector

2.3.1 Generational ZGC

2.4 Java Reference Types

2.4.1 WeakReference and ReferenceQueue

2.5 Reference Processing Pipeline

2.5.1 Discovery

2.5.2 Processing

2.5.3 Enqueue

2.6 Related Work

2.7 Summary

3. Method

3.1 Implementation Approach

3.1.1 ZGC Modifications

3.1.2 G1GC Modifications

3.2 Optimization Strategies

3.2.1 Null-Queue Skip Optimization

3.2.2 Data Structure Optimizations

3.2.3 Synchronization Optimizations

3.3 Benchmarking Methodology

3.3.1 Synthetic Benchmarks

3.3.2 Real-World Benchmarks

3.4 Experimental Setup

3.4.1 Hardware Configuration

3.4.2 Software Configuration

3.4.3 Measurement Methodology

4. Results

4.1 Synthetic Benchmark Results

4.1.1 Baseline Performance

4.1.2 Null-Queue Optimization Impact

4.1.3 Data Structure Impact

4.2 Caffeine Benchmark Results

4.2.1 Cache Operations Performance

4.2.2 GC Pause Time Analysis

4.3 Statistical Analysis

4.4 Summary of Findings

5. Discussion

5.1 Interpretation of Results

5.1.1 Performance Improvements

5.1.2 Trade-offs

5.2 Implications for JVM Development

5.2.1 Applicability to Other Collectors

5.2.2 Integration Challenges

5.3 Limitations

5.3.1 Benchmark Coverage

5.3.2 Correctness Considerations

5.3.3 Generational Complexity

5.4 Comparison with Related Work

5.5 Summary

6. Conclusion

6.1 Summary of Contributions

6.2 Answers to Research Questions

6.3 Future Work

6.3.1 Further Optimizations

6.3.2 Broader Evaluation

6.3.3 Integration with Mainline OpenJDK

6.4 Closing Remarks